Correction TP05

Matière : ATELIER DEVELOPPEMENT MOBILE AVANCE | **Classes : SEM31**

Exercice1

```
package com.lister.capteurs

class MainActivity : AppCompatActivity() {
    val TAG: String = "TP_Capteurs"
    var sensorManager: SensorManager? = null
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        setContentView(R.layout.activity_main)
        initApp()
    }
    private fun initApp() {
        // 1. Initialisation du SensorManager
        sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
        // 2. Récupération de la liste de tous les capteurs
        val sensorList = sensorManager!!.getSensorList(Sensor.TYPE_ALL)
        // 3. Affichage dans le Log.d avec un retour à la ligne
        Log.d(
            TAG, ("=== Liste des capteurs disponibles (" + sensorList.size + ") ===")
        )
        for (sensor:Sensor in sensorList) {
            Log.d(
                TAG, "Nom : " + sensor.getName() + " | Type : " + sensor.getStringType() +
                        " | Vendeur : " + sensor.getVendor()
            )
        }
    }
}
```

Exercice2

```
package com.affichage.acceleration

class MainActivity : AppCompatActivity(), SensorEventListener {
    companion object {
        private const val TAG = "TP_Accelero"
    }
    private lateinit var amb: ActivityMainBinding
    private lateinit var sensorManager: SensorManager
    private var accelerometer: Sensor? = null

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        initApp()
    }
```

```
    private fun initApp() {
        // 1. Initialisation du View Binding et affectation de la vue
        amb = ActivityMainBinding.inflate(layoutInflater)
        setContentView(amb.root)

        // 2. Initialisation des TextViews avec une valeur par défaut
        amb.txtX.text = "X : 0.0"
        amb.txtY.text = "Y : 0.0"
        amb.txtZ.text = "Z : 0.0"

        // 3. Initialisation du SensorManager et du capteur d'accélération
        sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
        accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)
    }
    override fun onResume() {
        super.onResume()
        // Exécute le bloc uniquement si l'accéléromètre n'est pas null
        // (évite un NullPointerException si le terminal n'intègre pas ce capteur)
        accelerometer?.let {
            sensorManager.registerListener(this, it, SensorManager.SENSOR_DELAY_NORMAL)
        }
    }
    override fun onPause() {
        super.onPause()
        // Désenregistrement dans onPause() pour couper le capteur
        // en arrière-plan et économiser la batterie
        sensorManager.unregisterListener(this)
    }

    override fun onSensorChanged(event: SensorEvent?) {
        // Traitement des variations de l'accéléromètre en temps réel
        if (event?.sensor?.type == Sensor.TYPE_ACCELEROMETER) {
            val x = event.values[0]
            val y = event.values[1]
            val z = event.values[2]

            // Mise à jour des TextViews avec les valeurs actuelles du capteur
            amb.txtX.text = "X : %.2f".format(x)
            amb.txtY.text = "Y : %.2f".format(y)
            amb.txtZ.text = "Z : %.2f".format(z)
        }
    }

    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {
        // Méthode obligatoire de l'interface SensorEventListener,
        // laissée vide ici
    }
}
```

Exercice3

```
package com.journal.local.acc

class MainActivity : AppCompatActivity(), SensorEventListener {

    companion object {
        private const val TAG = "TP_Accelero_SQL"
        private const val INTERVAL_ENREGISTREMENT_MS = 2000L // 2 secondes
    }

    private lateinit var amb: ActivityMainBinding
    private lateinit var sensorManager: SensorManager
    private var accelerometer: Sensor? = null
    private lateinit var sqliteMesure: SQLiteMesure

    // Variable pour gérer l'intervalle de 2 secondes
    private var dernierTempsEnregistrement: Long = 0

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        initApp()
    }

    private fun initApp() {
        // 1. Initialisation du View amb et affectation de la vue
        amb = ActivityMainBinding.inflate(layoutInflater)
        setContentView(amb.root)

        // 2. Initialisation de l'assistant SQLite SQLiteMesure
        sqliteMesure = SQLiteMesure(this,"Mesure01.db",null,1)

        // 3. Initialisation des TextViews avec une valeur par défaut
        amb.txtX.text = "X : 0.0"
        amb.txtY.text = "Y : 0.0"
        amb.txtZ.text = "Z : 0.0"

        // 4. Initialisation du SensorManager et du capteur d'accélération
        sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
        accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)

        ajouterEcouteur()
    }

    private fun ajouterEcouteur() {
        // 5. Gestion du clic sur le bouton pour lister les mesures dans le Logcat
        amb.btnListerMesures.setOnClickListener {
            sqliteMesure.listerMesures()
        }
    }

    override fun onResume() {
        super.onResume()
        // Exécute le bloc uniquement si l'accéléromètre n'est pas null
        // (évite un NullPointerException si le terminal n'intègre pas ce capteur)
        accelerometer?.let {
            sensorManager.registerListener(this, it, SensorManager.SENSOR_DELAY_NORMAL)
        }
    }

    override fun onPause() {
        super.onPause()
```

```
        // Désenregistrement dans onPause() pour couper le capteur
        // en arrière-plan et économiser la batterie
        sensorManager.unregisterListener(this)
    }

    override fun onSensorChanged(event: SensorEvent?) {
        // Traitement des variations de l'accéléromètre en temps réel
        if (event?.sensor?.type == Sensor.TYPE_ACCELEROMETER) {
            val x = event.values[0]
            val y = event.values[1]
            val z = event.values[2]

            // Mise à jour des TextViews avec les valeurs actuelles du capteur
            amb.txtX.text = "X : %.2f".format(x)
            amb.txtY.text = "Y : %.2f".format(y)
            amb.txtZ.text = "Z : %.2f".format(z)

            // Vérification si le Switch d'enregistrement est activé
            val isEnregistrementActif = amb.switchEnregistrement.isChecked

            if (isEnregistrementActif) {
                val tempsActuel = System.currentTimeMillis()
                // Vérifie si 2 secondes se sont écoulées depuis le dernier enregistrement
                if (tempsActuel - dernierTempsEnregistrement >= INTERVAL_ENREGISTREMENT_MS) {
                    dernierTempsEnregistrement = tempsActuel

                    // Récupération de la date et heure courante formatée
                    val dateFormat = SimpleDateFormat("yyyy-MM-dd HH:mm:ss", Locale.getDefault())
                    val dateTimeStr = dateFormat.format(Date())

                    // Insertion dans la base de données SQLite via SQLiteMesure
                    sqliteMesure.insererMesure(
                        typeCapteur = "ACCELEROMETER",
                        dateTime = dateTimeStr,
                        x = x,
                        y = y,
                        z = z
                    )
                }
            }
        }
    }

    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {
        // Méthode obligatoire de l'interface SensorEventListener,
        // laissée vide ici
    }
}

package com.votre.paquet

class SQLiteMesure(
    context: Context?,
    name: String?,
    factory: SQLiteDatabase.CursorFactory?,
    version: Int
) : SQLiteOpenHelper(context, name, factory, version) {
    companion object {
        private const val TAG = "SQLiteMesure"
        const val TABLE_NAME = "Mesure"
        const val COLUMN_ID = "id"
        const val COLUMN_TYPE_CAPTEUR = "type_capteur"
```

```
        const val COLUMN_DATE_TIME = "date_time"
        const val COLUMN_VAL_X = "valX"
        const val COLUMN_VAL_Y = "valY"
        const val COLUMN_VAL_Z = "valZ"
    }

    override fun onCreate(db: SQLiteDatabase?) {
        val createTableQuery = ("CREATE TABLE $TABLE_NAME ("
                + "$COLUMN_ID INTEGER PRIMARY KEY AUTOINCREMENT, "
                + "$COLUMN_TYPE_CAPTEUR TEXT, "
                + "$COLUMN_DATE_TIME TEXT, "
                + "$COLUMN_VAL_X REAL, "
                + "$COLUMN_VAL_Y REAL, "
                + "$COLUMN_VAL_Z REAL)")
        db?.execSQL(createTableQuery)
    }

    override fun onUpgrade(db: SQLiteDatabase?, oldVersion: Int, newVersion: Int) {
        db?.execSQL("DROP TABLE IF EXISTS $TABLE_NAME")
        onCreate(db)
    }

    // Méthode pour insérer une mesure
    fun insererMesure(typeCapteur: String, dateTime: String, x: Float, y: Float, z: Float) {
        val db = this.writableDatabase
        val values = ContentValues().apply {
            put(COLUMN_TYPE_CAPTEUR, typeCapteur)
            put(COLUMN_DATE_TIME, dateTime)
            put(COLUMN_VAL_X, x)
            put(COLUMN_VAL_Y, y)
            put(COLUMN_VAL_Z, z)
        }
        db.insert(TABLE_NAME, null, values)
        db.close()
    }

    // Méthode pour lister toutes les mesures dans les Logs
    fun listerMesures() {
        val db = this.readableDatabase
        val cursor = db.rawQuery("SELECT * FROM $TABLE_NAME", null)
        if (cursor.moveToFirst()) {
            do {
                val id = cursor.getInt(0)
                val type = cursor.getString(1)
                val date = cursor.getString(2)
                val x = cursor.getFloat(3)
                val y = cursor.getFloat(4)
                val z = cursor.getFloat(5)

                Log.d(TAG, "Mesure [$id] - Type: $type | Date: $date | X: $x, Y: $y, Z: $z")
            } while (cursor.moveToNext())
        } else {
            Log.d(TAG, "Aucune mesure enregistrée dans la base de données.")
        }
        cursor.close()
        db.close()
    }
}
```

Exercice4

```
package com.journal.distant.acc


class MainActivity : AppCompatActivity(), SensorEventListener {

    companion object {
        private const val INTERVAL_ENREGISTREMENT_MS = 2000L // 2 secondes
        private const val URL_INSERT = "http://172.16.41.248/JSON_MESURE/inserer_mesure.php"
        private const val URL_LIST = "http://172.16.41.248/JSON_MESURE/lister_mesures.php"
    }

    private lateinit var amb: ActivityMainBinding
    private lateinit var sensorManager: SensorManager
    private var accelerometer: Sensor? = null
    private lateinit var networkHelper: MesureNetworkHelper

    // Variable pour gérer l'intervalle de 2 secondes
    private var dernierTempsEnregistrement: Long = 0

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        initApp()
    }

    private fun initApp() {
        // 1. Initialisation du View Binding sous amb et affectation de la vue
        amb = ActivityMainBinding.inflate(layoutInflater)
        setContentView(amb.root)

        // 2. Instanciation de l'assistant réseau en lui passant le contexte
        networkHelper = MesureNetworkHelper(this)

        // 3. Initialisation des TextViews avec une valeur par défaut
        amb.txtX.text = "X : 0.0"
        amb.txtY.text = "Y : 0.0"
        amb.txtZ.text = "Z : 0.0"

        // 4. Initialisation du SensorManager et du capteur d'accélération
        sensorManager = getSystemService(SENSOR_SERVICE) as SensorManager
        accelerometer = sensorManager.getDefaultSensor(Sensor.TYPE_ACCELEROMETER)

        // 5. Configuration des écouteurs d'événements (boutons)
        ajouterEcouteur()
    }

    private fun ajouterEcouteur() {
        // Gestion du clic sur le bouton pour lister les mesures distantes via networkHelper
        amb.btnListerMesures.setOnClickListener {
            networkHelper.listerMesuresDistant(URL_LIST)
        }
    }

    override fun onResume() {
        super.onResume()
        // Exécute le bloc uniquement si l'accéléromètre n'est pas null
        accelerometer?.let {
            sensorManager.registerListener(this, it, SensorManager.SENSOR_DELAY_NORMAL)
        }
    }
```

```
    override fun onPause() {
        super.onPause()
        // Désenregistrement dans onPause() pour économiser la batterie
        sensorManager.unregisterListener(this)
    }

    override fun onSensorChanged(event: SensorEvent?) {
        if (event?.sensor?.type == Sensor.TYPE_ACCELEROMETER) {
            val x = event.values[0]
            val y = event.values[1]
            val z = event.values[2]

            // Mise à jour des TextViews avec les valeurs actuelles du capteur
            amb.txtX.text = "X : %.2f".format(x)
            amb.txtY.text = "Y : %.2f".format(y)
            amb.txtZ.text = "Z : %.2f".format(z)

            // Vérification si le Switch d'enregistrement est activé
            val isEnregistrementActif = amb.switchEnregistrement.isChecked

            if (isEnregistrementActif) {
                val tempsActuel = System.currentTimeMillis()
                // Vérifie si 2 secondes se sont écoulées depuis le dernier enregistrement
                if (tempsActuel - dernierTempsEnregistrement >= INTERVAL_ENREGISTREMENT_MS) {
                    dernierTempsEnregistrement = tempsActuel

                    // Récupération de la date et heure courante formatée
                    val dateFormat = SimpleDateFormat("yyyy-MM-dd HH:mm:ss", Locale.getDefault())
                    val dateTimeStr = dateFormat.format(Date())

                    // Envoi des données vers le serveur distant via l'instance de networkHelper
                    networkHelper.envoyerMesureDistant(URL_INSERT, "ACCELEROMETER", dateTimeStr, x, y,
z)
                }
            }
        }
    }

    override fun onAccuracyChanged(sensor: Sensor?, accuracy: Int) {
        // Méthode obligatoire, laissée vide ici
    }
}

package com.votre.paquet
class MesureNetworkHelper(private val context: Context) {
    private val TAG = "MesureNetworkHelper"
    // Envoi d'une mesure en POST avec gestion de la réponse du serveur
    fun envoyerMesureDistant(url: String, type: String, date: String, x: Float, y: Float, z: Float) {
        val stringRequest = object : StringRequest(
            Method.POST, url,
            { response ->
                try {
                    // Analyse de la réponse JSON renvoyée par le PHP
                    val jsonResponse = JSONObject(response)
                    val status = jsonResponse.getString("status")
                    val message = jsonResponse.optString("message", "")
                    if (status == "succes") {
                        Log.d(TAG, "Succès insertion : $message")
                    } else {
                        // Traitement de l'erreur renvoyée par le serveur (ex: paramètres manquants)
                        Log.e(TAG, "Erreur serveur (insertion) : $message")
                    }
                } catch (e: Exception) {
```

```
                Log.e(TAG, "Erreur de traitement JSON : ${e.message} | Réponse brute : $response")
            }
        },
        { error ->
            // Erreur réseau (pas de connexion, URL invalide, etc.)
            Log.e(TAG, "Erreur réseau lors de l'envoi : ${error.message}")
        }
    ) {
        override fun getParams(): Map<String, String> {
            val params = HashMap<String, String>()
            params["type_capteur"] = type
            params["date_time"] = date
            params["valX"] = x.toString()
            params["valY"] = y.toString()
            params["valZ"] = z.toString()
            return params
        }
    }
    Volley.newRequestQueue(context).add(stringRequest)
}
// Récupération de la liste en GET avec gestion des erreurs serveur
fun listerMesuresDistant(url: String) {
    val stringRequest = StringRequest(
        Request.Method.POST, url,
        { response ->
            try {
                val jsonResponse = JSONObject(response)
                val status = jsonResponse.getString("status")
                if (status == "succes") {
                    val dataArray = jsonResponse.getJSONArray("data")
                    if (dataArray.length() == 0) {
                        Log.d(TAG, "La base de données distante est vide.")
                    } else {
                        for (i in 0 until dataArray.length()) {
                            val item = dataArray.getJSONObject(i)
                            val id = item.getInt("id")
                            val type = item.getString("type_capteur")
                            val date = item.getString("date_time")
                            val x = item.getDouble("valX")
                            val y = item.getDouble("valY")
                            val z = item.getDouble("valZ")
                            Log.d(TAG,
                                "Distant [$id] - Type: $type | Date: $date | X: $x, Y: $y, Z: $z"
                            )
                        }
                    }
                } else {
                    val message = jsonResponse.optString("message", "Erreur inconnue")
                    Log.e(TAG, "Erreur serveur (liste) : $message")
                }
            } catch (e: Exception) {
                Log.e(TAG, "Erreur de traitement JSON (liste) : ${e.message}")
            }
        },
        { error ->
            Log.e(TAG, "Erreur réseau lors de la récupération : ${error.message}")
        }
    )

    Volley.newRequestQueue(context).add(stringRequest)
}
}
```